

Data Mining practice 9

Team 2

양승렬 2200409

최서현 22100133

문벼리 22000245

[Task1](#)

[Task2](#)

[Task3](#)

[Task4](#)

[Task5](#)

Task1

- alpha를 변경해서 학습을 수행하고 67페이지와 같은 그래프를 그려라

[row data]

이름	타입	설명
pop	double	인구
profit	double	수익

```
#데이터 불러오기
exData <- read.csv('ex1data1.txt', header = F, col.names = c(
exData
```

- 가설설정, cost함수, 각 theta 값을 계산하는 함수 생성

```
#가설 설정
h <- function(x, t0, t1){
  t0+t1*x
```

```

}

#cost함수
costJ <- function(t0, t1){
  m = nrow(exData)
  1/(2*m) * sum((h(exData$pop, t0, t1) - exData$profit) ** 2)
}

### 최적화 방법 : gradient_method

#theta0 : 절편
gradient_theta0 <-function(t0, t1){
  mean(h(exData$pop, t0, t1) -exData$profit)
}

#theta1 : 기울기
gradient_theta1 <-function(t0, t1){
  mean((h(exData$pop, t0, t1) -exData$profit) *exData$pop)
}

```

- alpha(0.001, 0.05)값을 초기에 다양하게 생성하여 벡터로 저장한 후, 각각에 대해 적용하며 학습을 확인

```

# alpha 값 설정
alpha_values <- c(0.001, 0.05)

# 초기 theta 값 설정
theta0 <- 0
theta1 <- 1
num_iter <- 1500

# 데이터를 저장할 데이터프레임 초기화
cost_data <- data.frame()

# 여러 alpha 값에 대해 반복 실행
for (alpha in alpha_values) {
  theta0 <- 0

```

```

theta1 <- 1
temp_costDF <- data.frame(iter = 0, t0 = theta0, t1 = theta1)

for (i in 1:num_iter) {
  theta0_update <- gradient_theta0(theta0, theta1)
  theta1_update <- gradient_theta1(theta0, theta1)

  theta0 <- theta0 - alpha * theta0_update
  theta1 <- theta1 - alpha * theta1_update

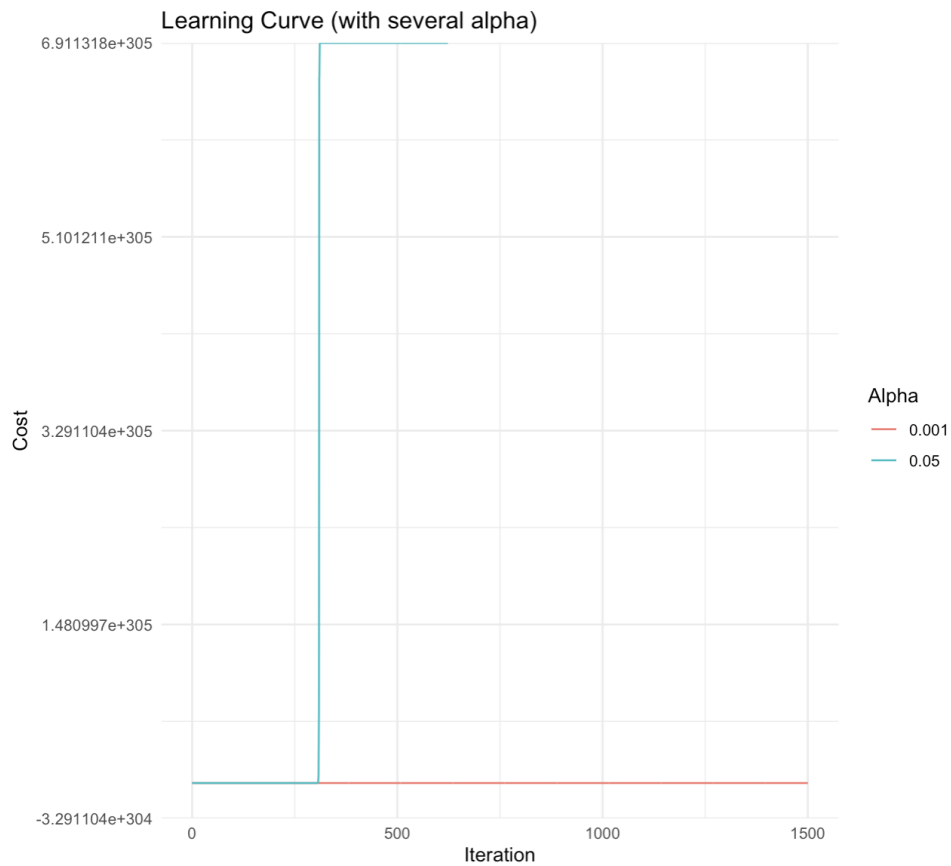
  # 각 반복마다 Cost 값 저장
  temp_costDF <- rbind(temp_costDF, c(i, theta0, theta1, cost))
}

cost_data <- rbind(cost_data, temp_costDF)
}

# 그래프 그리기
ggplot(cost_data, aes(x = iter, y = cost, color = as.factor(alpha))) +
  geom_line() +
  labs(title = "Learning Curve (with several alpha)", x = "Iteration", y = "Cost") +
  theme_minimal()

```

[Result]



```
summary(exData)
```

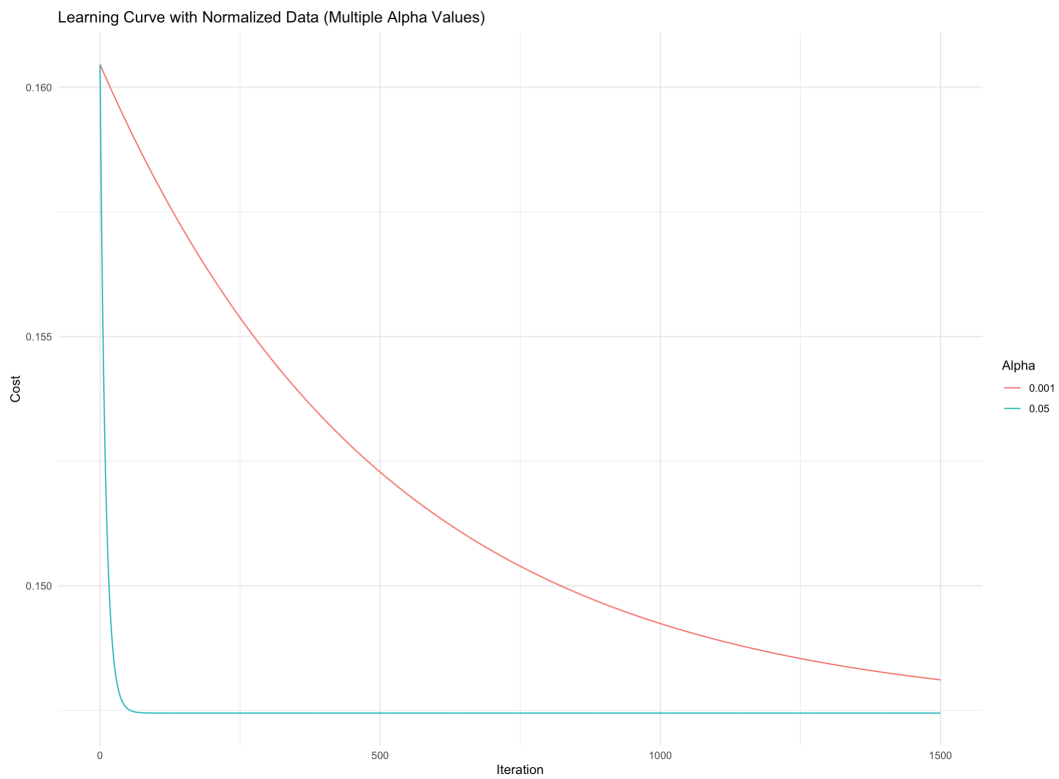
```
> summary(exData)
      pop      profit
Min.   : 5.027   Min.   : -2.681
1st Qu.: 5.708   1st Qu.:  1.987
Median : 6.589   Median :  4.562
Mean    : 8.160   Mean    :  5.839
3rd Qu.: 8.578   3rd Qu.:  7.047
Max.    :22.203   Max.    :24.147
```

표준화 이유

1. 안정적이고 일관성을 높임
2. 해석 용이

```
exData$pop <- scale(exData$pop)
```

[Result]



Task2

- Cost 감소가 멈추는 시점에서 학습을 중단하게끔 알고리즘을 수정하여라.
alpha값을 변경할 때, 학습에 소요된 iteration의 수(theta 업데이트 횟수)를 계산하여 표 혹은 그래프로 나타내어라.

1. cost감소가 멈추는 지점을 하나의 변수(tolerance)로 정리하고, 그 값 아래로 떨어지면 수렴했다고 판단하여 학습을 중단하도록 알고리즘 수정

```
# alpha 값 설정
alpha_values <- c(0.001, 0.005, 0.01, 0.05, 0.1)
tolerance <- 1e-4 # Cost 변화가 이 값 이하로 작아지면 수렴했다고 판단
max_iter <- 1500 # 최대 반복 횟수 제한

iteration_data <- data.frame(alpha = numeric(), iterations = )
# 각 alpha에 대해 경사 하강법 실행
```

```

for (alpha in alpha_values) {
  theta0 <- 0
  theta1 <- 1
  cost_prev <- costJ(theta0, theta1)

  for (i in 1:max_iter) {
    # 경사 하강법으로 theta0, theta1 업데이트
    theta0_update <- gradient_theta0(theta0, theta1)
    theta1_update <- gradient_theta1(theta0, theta1)

    theta0 <- theta0 - alpha * theta0_update
    theta1 <- theta1 - alpha * theta1_update

    # 새로운 Cost 계산
    cost_curr <- costJ(theta0, theta1)

    # 수렴 확인 (변화량이 tolerance보다 작아지면 중단)
    if (abs(cost_curr - cost_prev) < tolerance) {
      iteration_data <- rbind(iteration_data, data.frame(alpha, cost_curr, theta0, theta1))
      break
    }

    # 이전 Cost 값 갱신
    cost_prev <- cost_curr

    # 최대 반복 횟수까지 완료하면 수렴 실패로 간주
    if (i == max_iter) {
      iteration_data <- rbind(iteration_data, data.frame(alpha, cost_curr, theta0, theta1))
    }
  }
}

# iteration_data를 출력하여 확인
print(iteration_data)

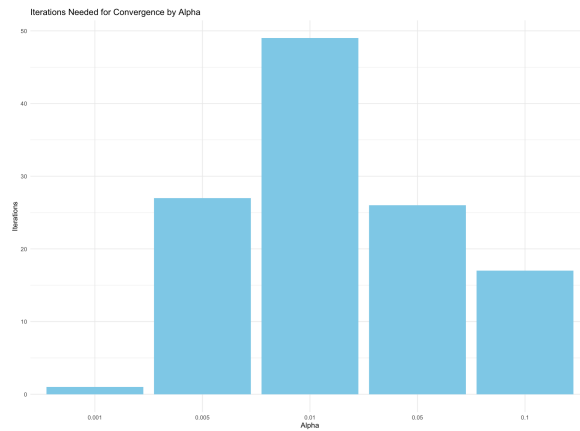
# iteration_data를 그래프로 시각화
ggplot(iteration_data, aes(x = as.factor(alpha), y = iteration_data$cost_curr)) +
  geom_bar(stat = "identity", fill = "skyblue") +

```

```
labs(title = "Iterations Needed for Convergence by Alpha",  
theme_minimal())
```

tolerance를 $1e-4$ 로 설정

```
> print(iteration_data)  
alpha iterations  
1 0.001          1  
2 0.005         27  
3 0.010         49  
4 0.050         26  
5 0.100         17
```



너무 작은 학습률로 tolerance보다 작은 경우 조기 종료: 0.001

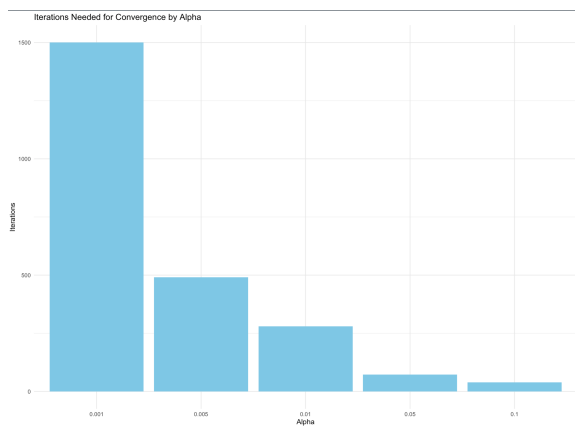
이후 0.01까지 학습률은 tolerance보다 작지 않아 많은 반복(계산)을 통해 최적점으로 수렴
0.01이후 학습률은 크기 때문에 빠르게 최적점에 도달해 반복횟수 감소

2. 문제점 발견

⇒ 너무 작은 학습률은 tolerance값보다 작아질 수 있기 때문에 tolerance값을 좀 더 보수적으로 설정

```
tolerance <- 1e-6
```

[Result]



alpaha	iterations
0.001	1500
0.005	491
0.01	280
0.05	72
0.1	39

3. 배운점

- 학습률이 낮으면 반복횟수가 높고 학습률이 높으면 반복횟수가 낮다는 것
- tolerance 보수적인 장점 : 수렴 조건이 보수적이기 때문에 좀 더 정확한 최적값 도출 가능

Task3

- 강의자료 65페이지의 <fill in> 부분을 완성하고 정상적으로 학습이 되는 것을 확인하여라. 67페이지와 같은 그래프를 완성하여라.

[raw data]

이름	타입	설명
size	integer	집 크기
num_bedroom	integer	방 개수
price	integer	가격

Linear Regression with multiple variables

```
exData2 <- read.csv("ex1data2.txt", header = F, col.names = c("size", "num_bedroom", "price"))
summary(exData2)
```



```
> summary(exData2)
      size      num_bedroom      price
Min.   : 852   Min.   :1.00   Min.   :169900
1st Qu.:1432   1st Qu.:3.00   1st Qu.:249900
Median :1888   Median :3.00   Median :299900
Mean   :2001   Mean   :3.17   Mean   :340413
3rd Qu.:2269   3rd Qu.:4.00   3rd Qu.:384450
Max.   :4478   Max.   :5.00   Max.   :699900
```

1. 표준화 cost계산

```
# 각 변수의 평균과 표준편차 계산
mean1 <- mean(exData2$size)
sd1 <- sd(exData2$size)
mean2 <- mean(exData2$num_bedroom)
sd2 <- sd(exData2$num_bedroom)
# 입력 변수들을 표준화 (평균 0, 표준편차 1로 변환)
exData2$size.norm <- (exData2$size - mean1) / sd1
exData2$num_bedroom.norm <- (exData2$num_bedroom - mean2) / sd2
# 입력 데이터 프레임 생성 및 편향(bias) 항목 추가
featureDF <- exData2 %>%
  mutate(bias = 1)%>%
  select(bias, ends_with('norm'))

#outcome Variable
labelVector <- exData2$price
theta_vector <- c(0,0,0)
# 가설 함수 정의
h <- function(x, theta_vector){
  theta_vector %*% x
}
# 비용 함수 정의
costFun <- function(theta_vector){
  v <- as.matrix(featureDF) %*% theta_vector - labelVector
  (t(v) %*% v / (2*nrow(featureDF)))[1,1]
}
costFun(theta_vector)
```

```

# 비용 함수와 파라미터 초기값을 저장할 데이터프레임 생성
costDF <- data.frame(iter = 0,
                     t0 = theta_vector[1],
                     t1 = theta_vector[2],
                     t2 = theta_vector[3],
                     cost = costFun(theta_vector))

# 반복 횟수, 학습률 및 데이터 개수 설정
num_iter = 1500
alpha <- 0.05
m <- nrow(featureDF)
# 경사 하강법을 사용하여 파라미터 업데이트
for( i in 1:num_iter){
  errors <- as.matrix(featureDF) %*% theta_vector - labelVect

  # 각 theta에 대한 기울기 계산
  theta_update <- alpha * (t(as.matrix(featureDF)) %*% errors)
  theta_vector <- theta_vector - theta_update
  costDF <- rbind(costDF, c(i, theta_vector, costFun(theta_vector)))
}

label_ko_num = function(num){
  ko_num = function(x){
    new_num = x %/% 10**10
    return(paste(new_num, 'x10^10', sep = ''))
  }
  return(sapply(num, ko_num))
}

#시각화
costDF %>% ggplot(aes(x = iter, y = cost))+
  geom_line()+
  scale_y_continuous(labels = label_ko_num)

```

2. Fill-in 설명

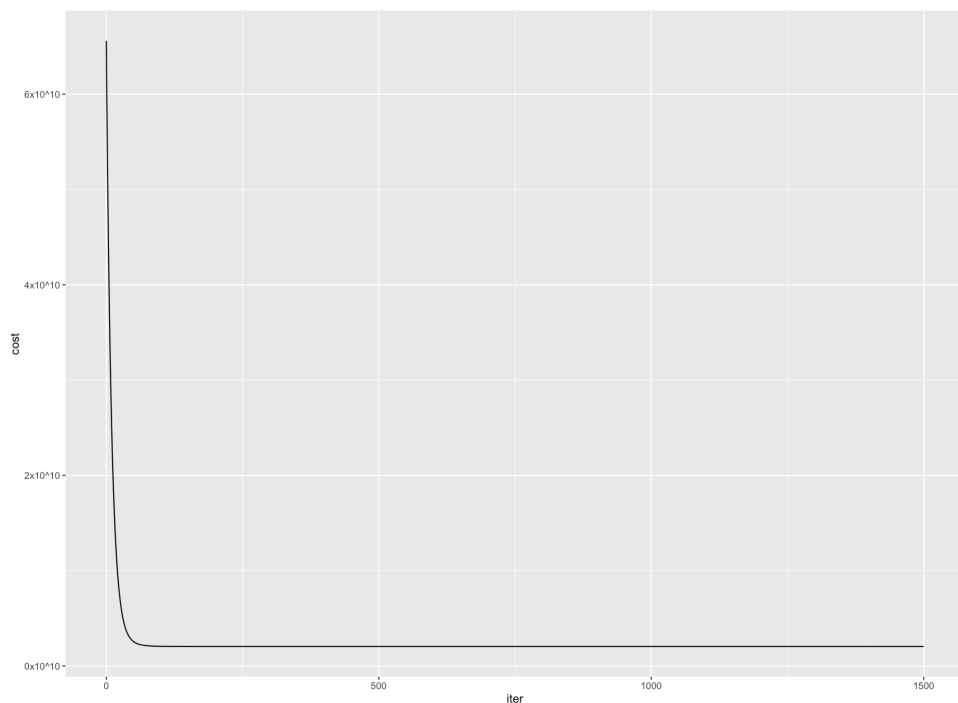
```

theta_update <- alpha * (t(as.matrix(featureDF)) %*% errors)

```

- `(t(as.matrix(featureDF)) %*% errors)` : featureDF 전치 후 에러와 내적
- `alpha` : 학습률을 곱하여 속도 조절
- `m` : feautrDF의 행 개수로 나누어 평균 theta로 도출

[Result]



3. 결과

- 초기에 급격하게 비용이 감소되고 빠르게 수렴

Task4

- 직접 구현한 선형회귀 코드를 사용해서 부동산 가격을 예측하는 선형회귀 모델을 학습 하여라

1. 데이터 확인

[row data]

이름	타입	설명
No	double	단순 인덱스
X1 transaction date	double	거래 날짜
X2 house age	double	주택 연식

X3 distance to the nearest MRT station	double	가장 가까운 지하철역 거리
X4 number of convenience stores	double	주변 편의점 수
X5 latitude	double	위도
X6 longitude	double	경도
Y house price of unit area	double	단위 면적당 주택 가격

[전처리]

1. 필요없는 열 제거 (NO)
2. feature name 변경

```
# 데이터 전처리
estate <- estate[, -1]
colnames(estate) <- c("transaction_date", "house_age", "dis
                      "num_convenience", "latitude", "longi
```

[최종 데이터 형식]

이름	타입	설명
transaction_date	double	거래 날짜
house_age	double	주택 연식
distance_to_mrt	double	가장 가까운 지하철역 거리
num_convenience	double	주변 편의점 수
latitude	double	위도
longitude	double	경도
price	double	단위 면적당 주택 가격

2. linear regression modeling

[전처리]

1. train, test 분할 (8:2)
 - `sample` 을 사용하여 랜덤으로 train, test split

- train : 331, test = 83

```
set.seed(2024)
n_sample <- nrow(estate)
sample_indices <- sample(1:nrow(estate), size = round(0.8

train <- estate[sample_indices, ]
test <- estate[-sample_indices, ]
```

2. 경사 하강법에 더욱 적합하도록 표준화 적용

$$X_{\text{norm}} = \frac{X - \mu}{\sigma}$$

- 데이터 표준화를 통해 평균을 0, 표준편차를 1로 변경
- 누수 방지를 위해 train, test 따로 진행

```
# 표준화를 위해 각각의 평균, 표준편차 저장
house_age_mean <- mean(train$house_age)
house_age_sd <- sd(train$house_age)

distance_to_mrt_mean <- mean(train$distance_to_mrt)
distance_to_mrt_sd <- sd(train$distance_to_mrt)

num_convenience_mean <- mean(train$num_convenience)
num_convenience_sd <- sd(train$num_convenience)

# train 표준화
train$house_age_norm <- (train$house_age - house_age_mean)
train$distance_to_mrt_norm <- (train$distance_to_mrt - distance_to_mrt_mean)
train$num_convenience_norm <- (train$num_convenience - num_convenience_mean)

# 필요 없는 열 제거
train <- train[, -c(1:3)]

# test 표준화
test$house_age_norm <- (test$house_age - house_age_mean) /
```

```
test$distance_to_mrt_norm <- (test$distance_to_mrt - dista
test$num_convenience_norm <- (test$num_convenience - num_c
```

```
# 필요 없는 열 제거
test <- test[, -c(1:3)]
```

```
# A tibble: 331 × 4
  price house_age_norm distance_to_mrt_norm num_convenience_norm
  <dbl>         <dbl>         <dbl>         <dbl>
1  44.3         1.90        -0.766         1.36
2  22.9        -0.305         0.629        -0.697
3  53.9        -1.35        -0.451        -0.0114
4   22        -0.810         1.51        -1.38
5  51.8        -1.49        -0.608         0.331
6  35.5         0.746        -0.490         0.331
7  34.1         1.15        -0.405        -0.697
8  30.5         1.26        -0.559         0.674
9   48        -0.401        -0.483         0.331
10  49        -1.49        -0.713         0.674
# i 321 more rows
# i Use `print(n = ...)` to see more rows
```

```
> test
# A tibble: 83 × 4
  price house_age_norm distance_to_mrt_norm num_convenience_norm
  <dbl>         <dbl>         <dbl>         <dbl>
1  47.3        -0.427        -0.429         0.331
2  43.1        -1.15        -0.561         0.331
3  39.3        -0.453        -0.483         0.331
4  37.4        -0.0448        -0.591        -1.04
5  47.7        -1.45        -0.844         1.02
6  51.6        -0.671        -0.647         1.02
7  56.2        -1.31        -0.566         0.331
8  57.1        -0.966        -0.514         0.331
9  46.2        -0.175        -0.639         0.331
10 34.7         1.55        -0.462         0.331
# i 73 more rows
# i Use `print(n = ...)` to see more rows
```

[모델 만들기]

- train을 통한 모델 학습

number of data (m)	331
number of feature (n)	4

1. 특징 행렬 생성

```
# train feature matrix 생성
train_featureDF <- train %>%
  mutate(bias = 1) %>%
  select(bias, house_age_norm, distance_to_mrt_norm, num_c

# test feature matrix 생성
test_featureDF <- test %>%
  mutate(bias = 1) %>%
  select(bias, house_age_norm, distance_to_mrt_norm, num_c
```

bias	house_age_norm	distance_to_mrt_norm	num_convenience_norm
1	1.901972855	-0.76637003	1.35922552
1	-0.305490884	0.62858769	-0.69669366
1	-1.348387139	-0.45081141	-0.01138727
1	-0.809557408	1.50900479	-1.38200005
1	-1.487439973	-0.60753934	0.33126593
1	0.746096173	-0.48987227	0.33126593
1	1.145873070	-0.40535341	-0.69669366
1	1.258853498	-0.55942676	0.67391913
1	-0.401089708	-0.48257642	0.33126593
1	-1.487439973	-0.71258109	0.67391913
1	0.085595211	-0.43232628	-0.01138727
1	1.441360343	-0.54959380	-1.03934686
1	-0.044766821	-0.51383660	1.35922552

- [m,n] 형태의 featureDF 생성

2. 목표변수 정의

```
train_labelVector <- train$price
test_labelVector <- test$price
```

3. theta 초기화

```
theta_vector <- rep(0, ncol(train_featureDF))
```

- [n,1] 형태의 theta_vector 생성

4. 가설 함수 정의

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

```
# 가설 함수 정의
h <- function(x, theta_vector) {
  as.matrix(x) %*% as.matrix(theta_vector, ncol = 1)
}
```

5. 비용 함수 정의

$$J(\theta_0, \theta_1, \dots, \theta_n) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

```
# Cost 함수 정의 (가설 함수 호출 포함)
costFun <- function(theta_vector) {
  predictions <- h(train_featureDF, theta_vector)
  errors <- predictions - train_labelVector
  (t(residuals) %*% errors / (2 * nrow(train_featureDF)))[1, ]
}
```

6. 경사하강법

$$\theta_n := \theta_n - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_n^{(i)}$$

```
# 하이퍼파라미터 정의
num_iter <- 1500
alpha <- 0.01

m <- nrow(train_featureDF)
```



```

# 모든 theta가 0인 경우의 비용함수
costDF <- data.frame(iter = 0, t(theta_vector), cost = cos

for (i in 1:num_iter) {
  # 기울기 계산
  predictions <- h(train_featureDF, theta_vector)
  errors <- predictions - train_labelVector
  theta_update <- alpha * (t(as.matrix(train_featureDF)) %

  # theta 업데이트
  theta_vector <- theta_vector - theta_update

  # Cost 값 기록
  costDF <- rbind(costDF, c(i, theta_vector, costFun(theta

}

```

columns	theta
bias	37.446817
house_age_norm	-2.791844
distance_to_mrt_norm	-6.377692
num_convenience_norm	4.374272

- 집 연식 → 음의 상관관계
 - 집이 새로울수록 가격이 높아지는 경향이 있음
- 지하철과의 거리 → 음의 상관관계 (짧을수록 좋음)
 - 지하철과의 거리가 짧을수록 가격이 높아지는 경향이 있음
- 주변 편의점의 수 → 양의 상관관계 (많을수록 좋음)
 - 주변 편의점이 많을수록 가격이 높아지는 경향이 있음

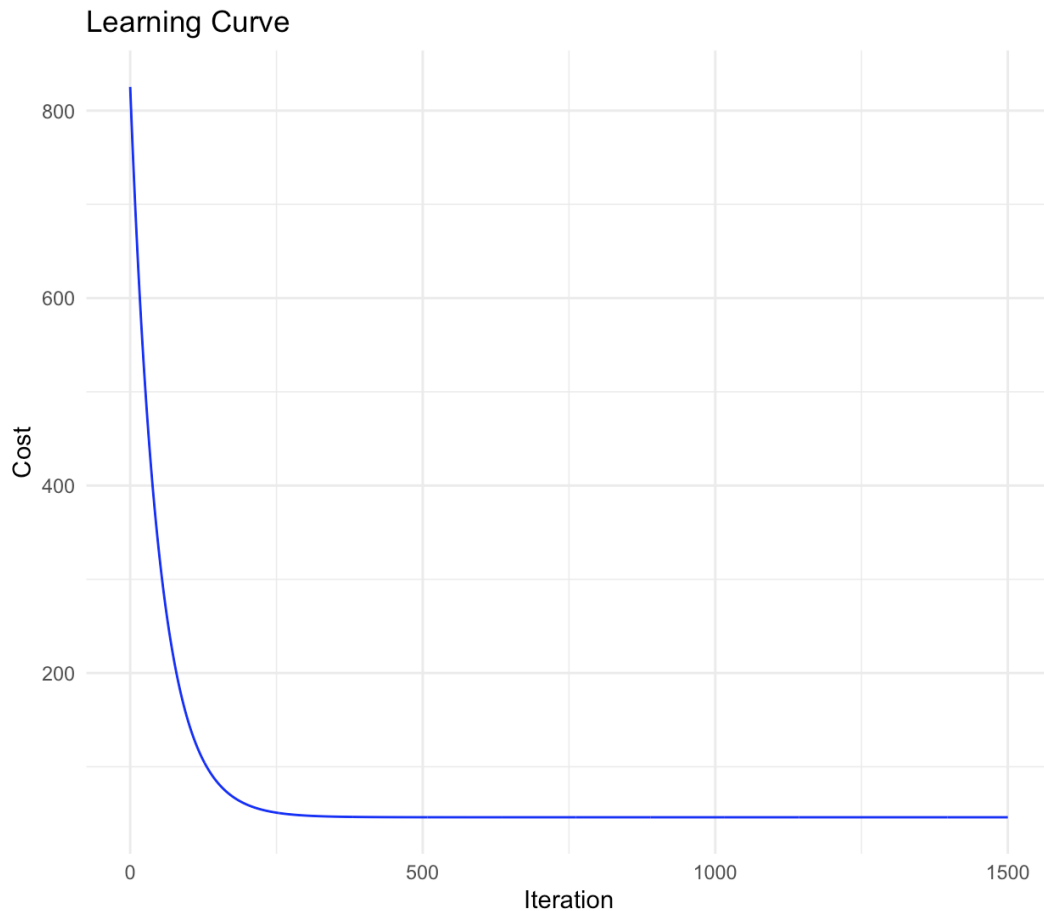
7. cost 변화 시각화

```

# Cost 값 시각화
ggplot(costDF, aes(x = iter, y = cost)) +

```

```
geom_line(color = "blue") +
labs(title = "Learning Curve", x = "Iteration", y = "Cost")
theme_minimal()
```



- iteration 250이하에서 cost가 수렴한 것을 확인

8. 예측 값 확인

```
# 예측 값 계산
train_pred <- as.matrix(train_featureDF) %*% theta_vector
test_pred <- as.matrix(test_featureDF) %*% theta_vector

### 성능 계산
### train
# MSE 계산
train_mse <- mean((train_pred - train_labelVector)^2)
```

```

# MAE 계산
train_mae <- mean(abs(train_pred - train_labelVector))

# R^2 계산
train_ss_total <- sum((train_labelVector - mean(train_labelVector))^2)
train_ss_residual <- sum((train_labelVector - train_pred)^2)
train_r_squared <- 1 - (train_ss_residual / train_ss_total)

# 성능 지표 출력
cat("train Mean Squared Error (MSE):", train_mse, "\n")
cat("train Mean Absolute Error (MAE):", train_mae, "\n")
cat("train R-squared:", train_r_squared, "\n")

#### test
# MSE 계산
test_mse <- mean((test_pred - test_labelVector)^2)

# MAE 계산
test_mae <- mean(abs(test_pred - test_labelVector))

# R^2 계산
test_ss_total <- sum((test_labelVector - mean(test_labelVector))^2)
test_ss_residual <- sum((test_labelVector - test_pred)^2)
test_r_squared <- 1 - (test_ss_residual / test_ss_total)

# 성능 지표 출력
cat("test Mean Squared Error (MSE):", test_mse, "\n")
cat("test Mean Absolute Error (MAE):", test_mae, "\n")
cat("test R-squared:", test_r_squared, "\n")

```

	MSE	MAE	R_squared
train	69.33039	6.277962	0.5945514
test	147.9093	7.283129	0.3669239

- train보다 test의 성능이 낮은 것을 확인할 수 있음
- 특히 MSE가 높은 것을 보아 테스트 데이터에 몇몇 큰 오차가 존재한다는 것을 알 수 있음

Task5

- lm함수를 사용해서 학습한 linear regression 모델과 Task 4 에서 얻은 linear regression 모델을 비교해 보자 학습한 모델을 비교해 보고 모델의 성능도 비교해보자

1. lm_model

[모델링]

1. 모델 만들기

```
lm_model <- lm(price ~ house_age_norm + distance_to_mrt_norm + num_convenience_norm, data = train)
```

- y = price
- X = house_age_norm, distance_to_mrt_norm, num_convenience_norm
- data = train

columns	theta_lm
bias	37.447
house_age_norm	-2.792
distance_to_mrt_norm	-6.380
num_convenience_norm	4.372

2. 예측

```
# 예측 값 계산
train_pred_lm <- predict(lm_model, newdata = train)
test_pred_lm <- predict(lm_model, newdata = test)

# 실제 값 저장
actual_train <- train$price
actual_test <- test$price
```

- predict 함수를 사용하여 예측

3. 성능 평가

```
#### 성능 계산
# train
# MSE 계산
train_mse_lm <- mean((train_pred_lm - actual_train)^2)

# MAE 계산
train_mae_lm <- mean(abs(train_pred_lm - actual_train))

# R^2 계산
train_ss_total_lm <- sum((actual_train - mean(actual_train))^2)
train_ss_residual_lm <- sum((actual_train - train_pred_lm)^2)
train_r_squared_lm <- 1 - (train_ss_residual_lm / train_ss_total_lm)

# 성능 지표 출력
cat("lm train Mean Squared Error (MSE):", train_mse_lm, "\n")
cat("lm train Mean Absolute Error (MAE):", train_mae_lm, "\n")
cat("lm train R-squared:", train_r_squared_lm, "\n")

# test
# MSE 계산
test_mse_lm <- mean((test_pred_lm - actual_test)^2)

# MAE 계산
test_mae_lm <- mean(abs(test_pred_lm - actual_test))

# R^2 계산
test_ss_total_lm <- sum((actual_test - mean(actual_test))^2)
test_ss_residual_lm <- sum((actual_test - test_pred_lm)^2)
test_r_squared_lm <- 1 - (test_ss_residual_lm / test_ss_total_lm)

# 성능 지표 출력
cat("lm test Mean Squared Error (MSE):", test_mse_lm, "\n")
cat("lm test Mean Absolute Error (MAE):", test_mae_lm, "\n")
cat("lm test R-squared:", test_r_squared_lm, "\n")
```

	MSE	MAE	R_squared
train_lm	69.33038	6.277891	0.5945515
test_lm	147.8977	7.282872	0.3669734

[비교]

1. theta

	Task4 model	Task5 model
bias	37.446817	37.447
house_age_norm	-2.791844	-2.792
distance_to_mrt_norm	-6.377692	-6.380
num_convenience_norm	4.374272	4.372

- 아주 미세한 차이가 있다.
- lm에서는 최적화 방법으로 경사하강법이 아닌 최소자승법을 사용한다.
- 최소자승법은 데이터의 잔차 제곱합을 최소화하여 최적의 회귀 계수를 구하는 방법이며 경사하강법과 달리 공식을 사용해 최적의 해를 바로 계산한다.

2. 성능

	Task4 model	Task5 model
trian_MSE	69.33039	69.33038
trian_MAE	6.277962	6.277891
trian_R2	0.5945514	0.5945515
test_MSE	147.9093	147.8977
test_MAE	7.283129	7.282872
test_R2	0.3669239	0.3669734

- 두 모델의 결과값은 거의 동일한 것을 확인함